

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ**  
**ΤΜΗΜΑ ΜΗΧ Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ**  
**ΤΟΜΕΑΣ ΥΛΙΚΟΥ ΚΑΙ ΑΡΧΙΤΕΚΤΟΝΙΚΗΣ ΥΠΟΛΟΓΙΣΤΩΝ**  
**Εργαστήριο Ηλεκτρονικών & Λογικού Σχεδιασμού**

**ΕΡΓΑΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ**  
**ΛΟΓΙΚΟΥ ΣΧΕΔΙΑΣΜΟΥ**

**Γ.Φ. ΑΛΕΞΙΟΥ - Ε.Ζ. ΨΑΡΑΚΗΣ**

**Πάτρα 2018**

**Ακαδ. Έτος : 2020-21**

**Εαρινό Εξάμηνο : 2020-21**

**Ονοματεπώνυμο: ΚΙΟΥΡΤ-ΜΕΧΜΕΤΑΛΗ ΑΧΜΕΤ**

**Α.Μ. : 1084672**

**Ονομ/νυμο Συνεργάτη:\_\_\_\_\_**

**Α.Μ. : \_\_\_\_\_**

**Ονομ/νυμο Συνεργάτη:\_\_\_\_\_**

**Α.Μ. : \_\_\_\_\_**

**Τμήμα : \_\_\_\_\_**

**Άσκηση: \_\_ ΑΣΚΗΣΗ 5 \_\_**

***Αναλυτική Βαθμολογία Πρακτικού Μέρους:***

|                    | <b>Βαθμός<br/>Αριθ.</b> | <b>Αξιολογητή<br/>Ολογραφ.</b> | <b>Υπογραφή Αξιολογητή</b> |
|--------------------|-------------------------|--------------------------------|----------------------------|
| <i>Άσκηση 1</i>    |                         |                                |                            |
| <i>Άσκηση 3</i>    |                         |                                |                            |
| <i>Άσκηση 5</i>    |                         |                                |                            |
| <i>Άσκηση 6</i>    |                         |                                |                            |
| <i>Άσκηση 7</i>    |                         |                                |                            |
| <b><i>Μ.Ο.</i></b> |                         |                                |                            |

**ΑΣΚΗΣΗ 5****ΗΜΕΡΟΜΗΝΙΑ:****ΘΕΜΑ: ΑΡΙΘΜΗΤΙΚΗ\_ΛΟΓΙΚΗ ΜΟΝΑΔΑ  
(ARITHMETIC LOGIC UNIT, ALU)****ΘΕΩΡΙΑ****ALU**

Η ALU είναι ένα συνδυαστικό κύκλωμα δύο εισόδων 4 bits που μπορεί να εκτελέσει διάφορες αριθμητικές και λογικές πράξεις. Το είδος της πράξης καθορίζεται από τον συνδυασμό που θα δώσουμε στις εισόδους επιλογής ( $S_3, S_2, S_1, S_0, M, C_n$ ). Ο παρακάτω πίνακας συνοψίζει τις διαθέσιμες λειτουργίες της ALU (συμβουλευτείτε επίσης το παράρτημα για το 74181).

**ΠΡΟΣΟΧΗ: Η είσοδος  $C_n$  της ALU είναι αντεστραμμένη (active low!) που σημαίνει ότι όταν  $C_n=0$  τότε νοιάργει κρατούμενο στην είσοδο και όταν  $C_n=1$  τότε δεν λαμβάνεται υπόψη (Δείτε στο Παράρτημα, ποδαράκι 7)**

**ΠΡΟΣΟΧΗ: Η έξοδος  $C_{n+1}$  της ALU είναι αντεστραμμένη (active low!) που σημαίνει ότι όταν από τη πράξη προκύπτει κρατούμενο τότε  $C_{n+1}=0$  και όταν δεν προκύπτει τότε  $C_{n+1}=1$  (Δείτε στο Παράρτημα, ποδαράκι 16)**

**Πίνακας 1.** Διαθέσιμες Λειτουργίες του 74LS181

| $S_3S_2S_1S_0$ | $M=1$            | $M=0, C_n=1,$                             | $M=0, C_n=0$                                    |
|----------------|------------------|-------------------------------------------|-------------------------------------------------|
| 0 0 0 0        | $F = A'$         | $F = A$                                   | $F = A \text{ PLUS } 1$                         |
| 0 0 0 1        | $F = (A + B)'$   | $F = A + B$                               | $F = (A + B) \text{ PLUS } 1$                   |
| 0 0 1 0        | $F = A'B$        | $F = A + B'$                              | $F = (A + B') \text{ PLUS } 1$                  |
| 0 0 1 1        | $F = 0$          | $F = \text{MINUS } 1$                     | $F = \text{ZERO}$                               |
| 0 1 0 0        | $F = (AB)'$      | $F = A \text{ PLUS } AB'$                 | $F = A \text{ PLUS } AB' \text{ PLUS } 1$       |
| 0 1 0 1        | $F = B'$         | $F = (A + B) \text{ PLUS } AB'$           | $F = (A + B) \text{ PLUS } AB' \text{ PLUS } 1$ |
| 0 1 1 0        | $F = A \oplus B$ | $F = A \text{ MINUS } B \text{ MINUS } 1$ | $F = A \text{ MINUS } B$                        |
| 0 1 1 1        | $F = AB'$        | $F = AB' \text{ MINUS } 1$                | $F = AB'$                                       |
| 1 0 0 0        | $F = A' + B$     | $F = A \text{ PLUS } AB$                  | $F = A \text{ PLUS } AB \text{ PLUS } 1$        |
| 1 0 0 1        | $F = A \odot B$  | $F = A \text{ PLUS } B$                   | $F = A \text{ PLUS } B \text{ PLUS } 1$         |
| 1 0 1 0        | $F = B$          | $F = (A + B') \text{ PLUS } AB$           | $F = (A + B') \text{ PLUS } AB \text{ PLUS } 1$ |
| 1 0 1 1        | $F = AB$         | $F = AB \text{ MINUS } 1$                 | $F = AB$                                        |
| 1 1 0 0        | $F = 1$          | $F = A \text{ PLUS } A^\dagger$           | $F = A \text{ PLUS } A \text{ PLUS } 1$         |
| 1 1 0 1        | $F = A + B'$     | $F = (A + B) \text{ PLUS } A$             | $F = (A + B) \text{ PLUS } A \text{ PLUS } 1$   |
| 1 1 1 0        | $F = A + B$      | $F = (A + B') \text{ PLUS } A$            | $F = (A + B') \text{ PLUS } A \text{ PLUS } 1$  |
| 1 1 1 1        | $F = A$          | $F = A \text{ MINUS } 1$                  | $F = A$                                         |

Όπου “+”  $\rightarrow$  OR, “ $\oplus$ ”  $\rightarrow$  XOR, “ $\odot$ ”  $\rightarrow$  XNOR, “PLUS”  $\rightarrow$  αριθμ. πρόσθεση, “MINUS”  $\rightarrow$  αριθμ. αφαίρεση

## ΔΙΑΔΙΚΑΣΙΑ

**ΠΡΟΣΟΧΗ :** Πριν ξεκινήσετε την υλοποίηση της άσκησης κατεβάστε ΟΠΩΣΔΗΠΟΤΕ από το eclass του εργαστηρίου (από την κατηγορία “Έγγραφα\Logisim-Evolution”) το .zip αρχείο με την ανανεωμένη βιβλιοθήκη για το Logisim (**CEID\_LogicDesign\_LogisimEV\_library\_v1.1**). Για το πως εισάγουμε βιβλιοθήκη στο Logisim-Ev, συμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα “Έγγραφα/Logisim-Evolution” του eclass του εργαστηρίου.

1. Συμπληρώστε τον παρακάτω πίνακα, θέτοντας τις εισόδους επιλογής ( $S_3, S_2, S_1, S_0, M, C_n$ ) της ALU (74181) στις κατάλληλες τιμές, (συμβουλευτείτε τον Πίνακα 1), και ελέγξτε τις αντίστοιχες αριθμητικές και λογικές λειτουργίες της ALU δίνοντας τα κατάλληλα screenshots.

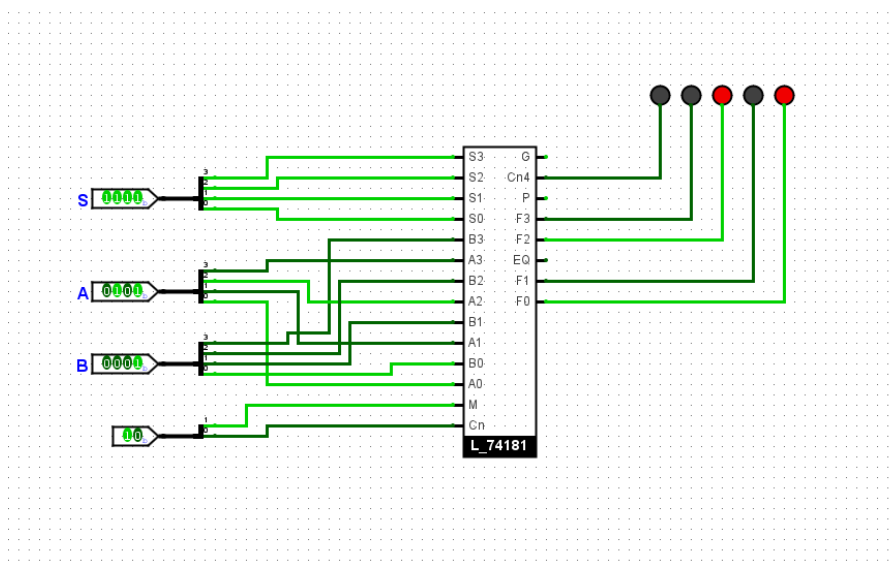
| Πράξη ALU                | $S_3$ | $S_2$ | $S_1$ | $S_0$ | $M$ | $C_n$ |
|--------------------------|-------|-------|-------|-------|-----|-------|
| $F = A$                  | 1     | 1     | 1     | 1     | 1   | X     |
| $F = B$                  | 1     | 0     | 1     | 0     | 1   | X     |
| $F = A \text{ XNOR } B$  | 1     | 0     | 0     | 1     | 1   | X     |
| $F = A \text{ MINUS } B$ | 0     | 1     | 1     | 0     | 0   | 0     |
| $F = A \text{ PLUS } B$  | 1     | 0     | 0     | 1     | 0   | 1     |

## Παρατηρήσεις:

- Χρησιμοποιείτε το ολοκληρωμένο 74181 της βιβλιοθήκης **CEID\_LogicDesign\_LogisimEV**
- Συνδέστε τα σήματα επιλογής  $S_3, S_2, S_1, S_0, M, C_n$  σε pins εισόδου και αποδώστε τιμές σε αυτά βάση του παραπάνω πίνακα που συμπληρώσατε
- Συνδέστε τα A, B σε pins εισόδου και αποδώστε τις τιμές  $A = 5_{10}$  και  $B = 1_{10}$
- Συνδέστε τις εξόδους,  $nC_{n4}, Y_4, Y_3, Y_2, Y_1$ , σε LEDs για να επιβεβαιωθείτε τη εκάστοτε λειτουργία της ALU.
- **ΠΡΟΣΟΧΗ: Η είσοδος  $C_n$  της ALU είναι αντεστραμμένη (active low!). Η έξοδος  $C_{n4}$  της ALU είναι αντεστραμμένη (active low!) (δείτε στο Παράτημα, ποδαράκια 7, 16)**

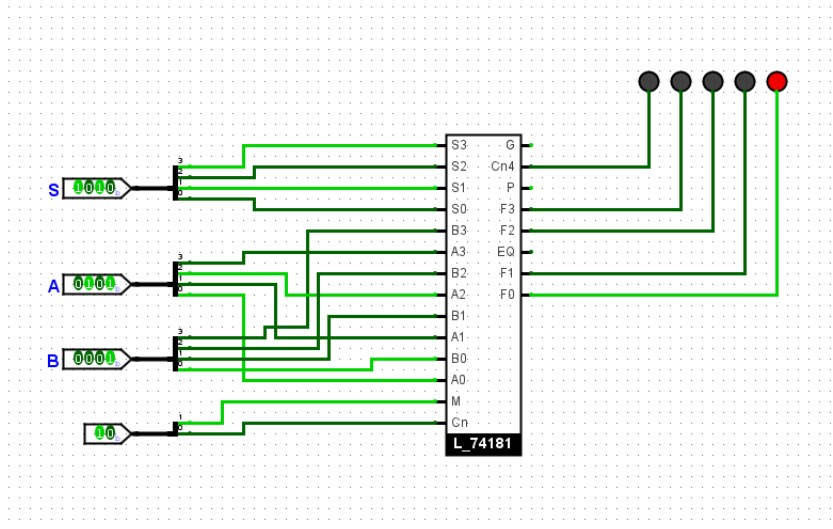
## ΚΥΚΛΩΜΑ

- Screenshot για την πράξη  $F=A$  ( $A = 5_{10}$ ,  $B=1_{10}$ )

*Σύντομη επεξήγηση του αποτελέσματος*

Για να έχουμε στην έξοδο μας μόνο το A πρέπει πρώτα συμφώνα με το πίνακα μας παραπάνω να δηλώνουμε στο  $S = 1111$  και το  $Cn$  στην συγκεκριμένη περίπτωση δεν έχει καμία σημασία λόγω του  $M=1$  οπότε δεν αλλάζει το αποτέλεσμα αν έχει ή δεν έχει κρατούμενο. Επίσης και στην έξοδο δεν έχει καμία σημασία το  $Cn4$  επειδή δεν χρειάζεται κάποιο κρατούμενο στην έξοδο. Εκτός από αυτό δεν χρειάζεται και δεν έχει κάποια σημασία το κρατούμενο λόγω της πράξης που κάνουμε που είναι μεταφορά ενός input. Τέλος, βλέπουμε ότι στην έξοδο έχουμε τις τιμές A.

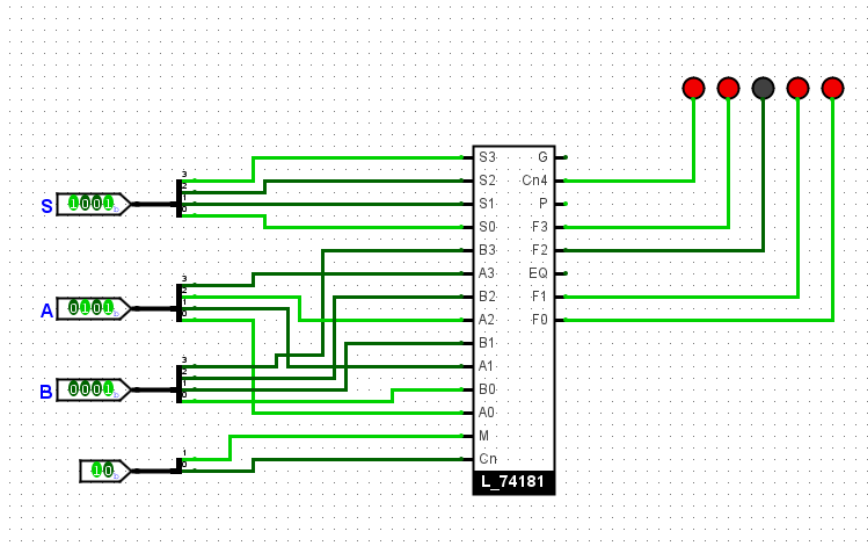
- Screenshot για την πράξη  $F=B(A = 5_{10}, B=1_{10})$



#### *Σύντομη επεξήγηση του αποτελέσματος*

Για να βγάλουμε τις τιμές του B σύμφωνα με την εκφώνηση βάζουμε στο S=1010. Ξανά δεν έχει καμία σημασία το Cn λόγω του M=1. Τέλος, και το Cn4 δεν έχει καμία σημασία επειδή κάνουμε μία πράξη μεταφοράς.

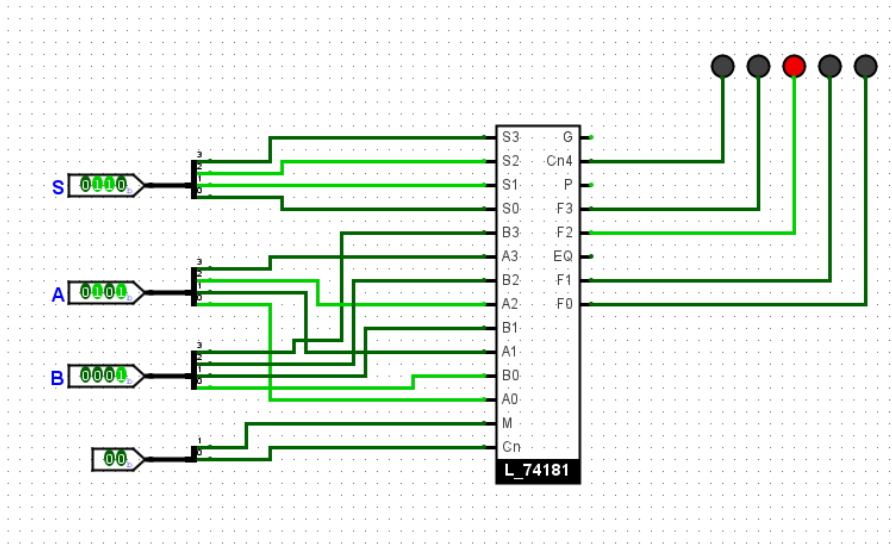
- Screenshot για την πράξη  $F=A \text{ XNOR } B$  ( $A = 5_{10}, B=1_{10}$ )



#### Σύντομη επεξήγηση του αποτελέσματος

Για να λειτουργήσει το κύκλωμα ως XNOR κάνουμε το  $S=1001$  και  $M=1$  όπως λέει και ο πίνακας μας. Δεν έχει σημασία το  $Cn$  λόγω του  $M$ . Ανάβει το LED του  $Cn4$  που σημαίνει δεν έχει κάποιο κρατούμενο το  $Cn4$  σύμφωνα με την εκφώνηση μας.

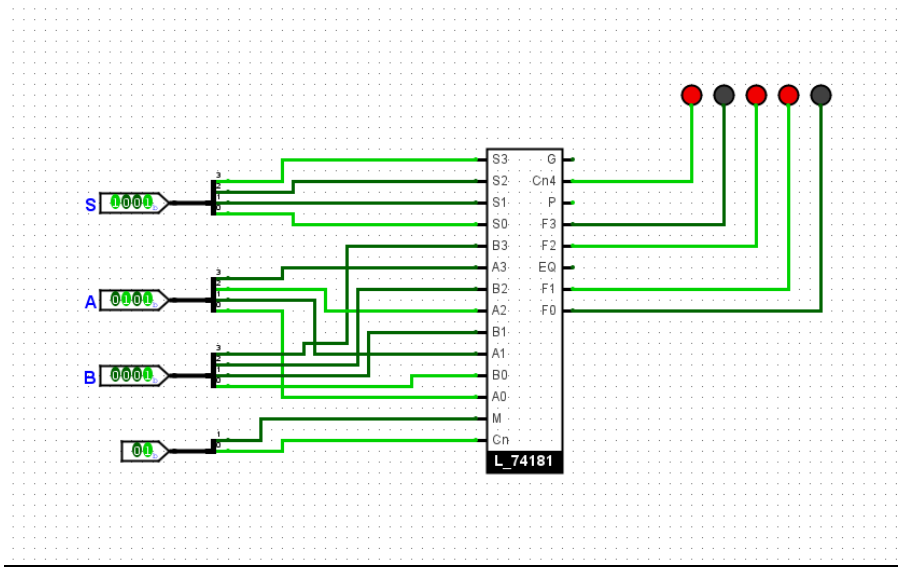
- Screenshot για την πράξη  $F=A \text{ MINUS } B$  ( $A = 5_{10}, B=1_{10}$ )



**Σύντομη επεξήγηση του αποτελέσματος**

Πρώτα για την πράξη  $S=0110$ . Η μεγαλύτερη διαφορά που μπορούμε να δούμε αυτή την φορά είναι ότι έχουμε το  $M=0$  για να έχουμε το κρατούμενο  $C_n$  της εισόδου για να κάνουμε την πράξη MINUS σύμφωνα με την εκφώνηση. Επειδή κάνουμε αφαίρεση το 0001 μετατρέπεται από 2's complement σε 1111 που όταν το προσθέτουμε με την A παίρνουμε το αποτέλεσμα 10100 που το κρατούμενο στο  $C_n4$  δεν ανάβει αρά είναι 1 σύμφωνα με την άσκηση μας.

- Screenshot για την πράξη  **$F=A \text{ PLUS } B$**  ( $A = 5_{10}$ ,  $B=1_{10}$ )

**Σύντομη επεξήγηση του αποτελέσματος**

Για την πράξη  $S=1001$  που έχουμε το  $M=0$  και το κρατούμενο εισόδου  $C_n$  για να κάνουμε την πράξη PLUS σύμφωνα με την εκφώνηση. Κάνουμε την πρόθεση των δύο τιμών που το κρατούμενο εξόδου είναι 0 διότι ανάβει το LED. Χωρίς το κρατούμενο έχουμε την τιμή εξόδου 0110.



2. Χρησιμοποιώντας την ALU (74181) και τα κατάλληλα ολοκληρωμένα, υλοποιείστε στον simulator συνδυαστικό κύκλωμα το οποίο να συγκρίνει δύο αριθμούς των τεσσάρων bits, δικαιολογώντας πλήρως την απάντησή σας και δίνοντας τα απαραίτητα screenshots. Οι τελικές εξοδοι του κυκλώματός σας θα είναι 3: **G, E, L** και θα παίρνουν τις τιμές G=1 μόνο όταν A>B, E=1 μόνο όταν A=B και L=1 μόνο όταν A<B. Ισοδύναμα

|     | Έξοδοι (1-bit) |           |          |
|-----|----------------|-----------|----------|
|     | G (Greater)    | E (Equal) | L (Less) |
| A>B | 1              | 0         | 0        |
| A=B | 0              | 1         | 0        |
| A<B | 0              | 0         | 1        |

#### Παρατηρήσεις:

- Από την βιβλιοθήκη **CEID\_LogicDesign\_LogisimEV** χρησιμοποιείτε **MONO** το ολοκληρωμένο 74181. Από την βιβλιοθήκη **TTL** μπορείτε να χρησιμοποιήσετε ολοκληρωμένα της επιλογής σας.
- Για την σχεδίαση του κυκλώματος σύγκρισής λαμβάνετε υπόψη σας **MONO** τις εξόδους C<sub>n4</sub>, F<sub>3</sub>, F<sub>2</sub>, F<sub>1</sub>, F<sub>0</sub>, της ALU.
- ΠΡΟΣΟΧΗ: Η είσοδος C<sub>n</sub> της ALU είναι αντεστραμμένη (active low!). Η έξοδος C<sub>n4</sub> της ALU είναι αντεστραμμένη (active low!) (δείτε στο Παράτημα, ποδαράκια 7, 16)**
- Συνδέστε τα σήματα επιλογής S<sub>3</sub>, S<sub>2</sub>, S<sub>1</sub>, S<sub>0</sub>, M, C<sub>n</sub> σε pins εισόδου και αποδώστε τιμές σε αυτά για την πράξη που θεωρείτε ότι πρέπει να γίνει (σύμφωνα με τον Πίνακα 1 της θεωρίας). Συνδέστε επίσης τις εισόδους A, B σε pins εισόδου και τις εξόδους του κυκλώματός σας (G, E, L) σε LEDs για να επιβεβαιώσετε τη σωστή λειτουργία του κυκλώματος που σχεδιάσατε. *Υπόδειξη: μπορείτε να συνδέσετε και τις εξόδους τις ALU (C<sub>n4</sub>, F<sub>3</sub>, F<sub>2</sub>, F<sub>1</sub>, F<sub>0</sub>) σε LEDs για να παρατηρείτε κάθε φορά το αποτέλεσμα της ALU*
- Σε περίπτωση σχεδίασης υποκυκλώματος πρέπει να συμπεριλάβετε και ξεχωριστό screenshot για το υποκύκλωμα που χρησιμοποιείτε!!

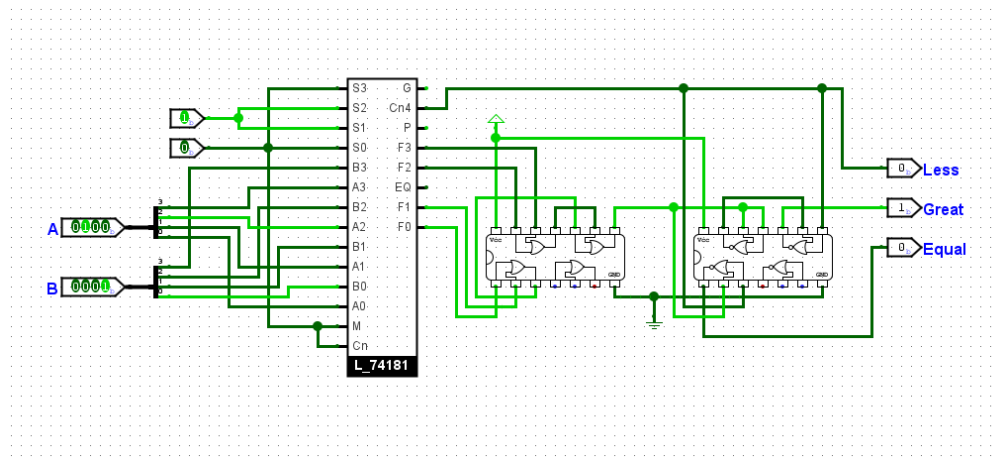
## ΣΥΝΔΙΑΣΤΙΚΟ ΚΥΚΛΩΜΑ

### Σύντομη δικαιολόγηση της επιλογής σχεδίασης

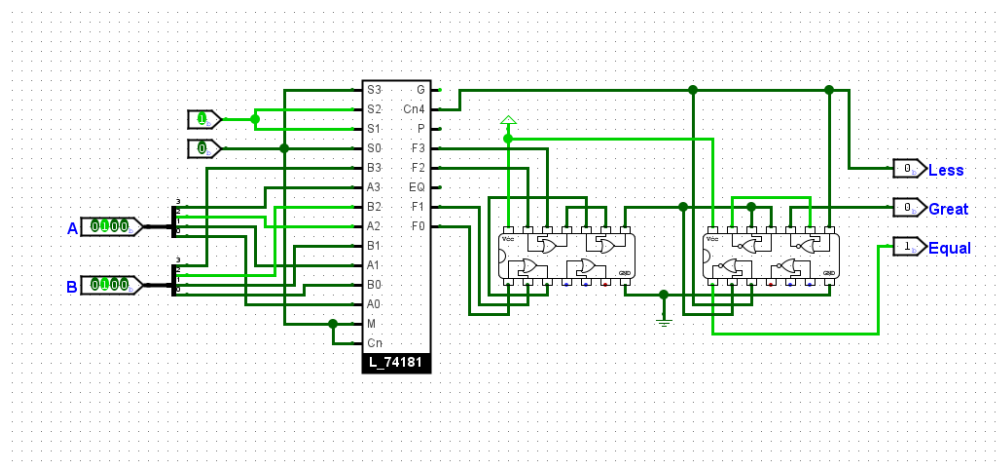
Όπως κάναμε και στην προηγούμενη άσκηση συνδέουμε τα input A και B. Που για αυτή την φορά τις χρησιμοποιούμε για σύγκριση των 2 αυτών αριθμών. Στη συνέχεια βλέπουμε ότι συνδέουμε όλα τα F (F0,F1,F2,F3) σε chip OR ώστε αυτά να συνδέσουν σε chip NOR για να μπορούμε να κάνουμε την σύγκριση. Κάνουμε τις λογικές πράξεις και για τις 3 περιπτώσεις χρησιμοποιώντας και το Vcc για να έχουμε τα αποτελέσματα που χρειάζονται.

Επειδή το αποτέλεσμα του κυκλώματος βγαίνει σε 2's complement, για το L αρκεί να δούμε πότε το most significant bit Cn4 είναι 1, συνεπώς  $L = Cn4$ . Για το Great θες ένα από τα bits F3-F0 1 και ταυτόχρονα το Cn4 0, άρα  $G = Cn4'(F3 + F2 + F1 + F0)$ . Τέλος, για την Equal θέλεις όλα τα bits 0, άρα  $E = Cn4'F3'F2'F1'F0'$ .

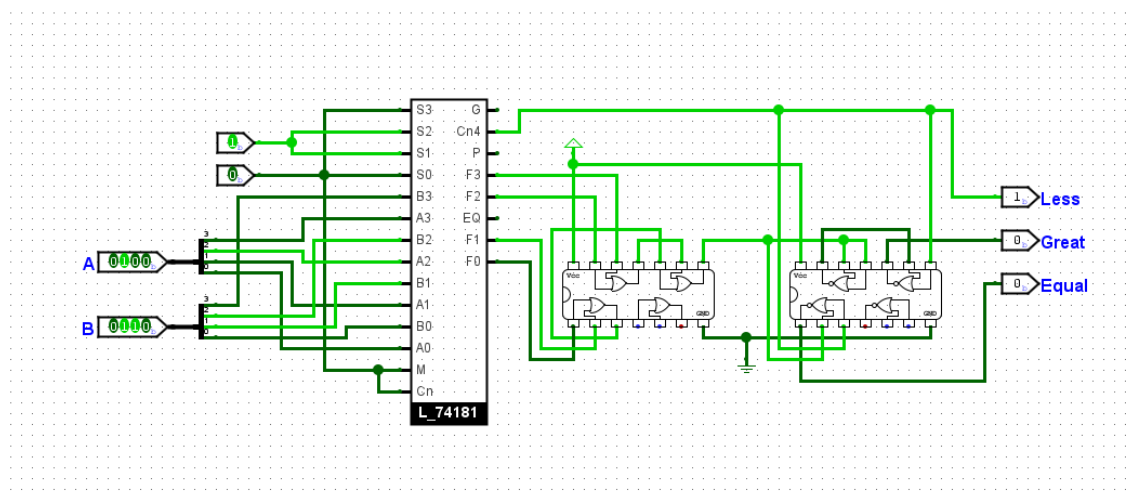
- Screenshot για την περίπτωση  $A = 4_{10}$ ,  $B = 1_{10}$



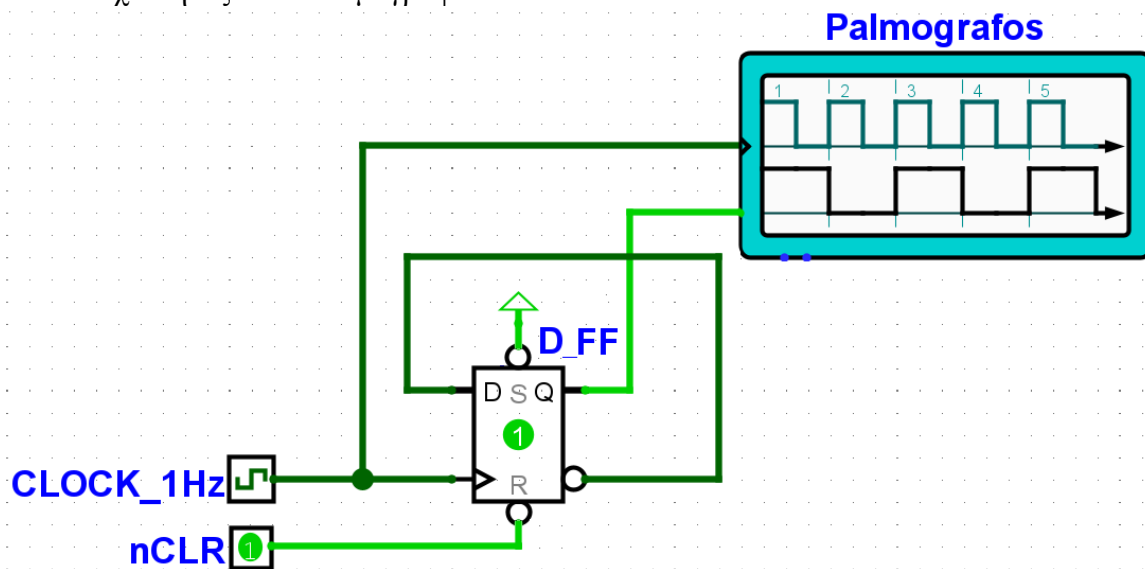
- Screenshot για την περίπτωση  $A = 4_{10}$ ,  $B=4_{10}$



- Screenshot για την περίπτωση  $A = 4_{10}$ ,  $B=6_{10}$



**3.Α.** Το κύκλωμα του Σχήματος 1 χρησιμοποιεί ένα D Flip-Flop για να παράγει έναν τετραγωνικό παλμό στη έξοδο του, Q, με συχνότητα  $f_1 = f_0/2$ , όπου  $f_0$  η συχνότητα του του τετραγωνικού παλμού στην είσοδο CLOCK του Flip-Flop. Υλοποιήστε στον simulator το κύκλωμα του Σχήματος 1 **χρησιμοποιώντας το ολοκληρωμένο 74LS74** και επιβεβαιώστε τη διαίρεση συχνότητας στον παλμογράφο.



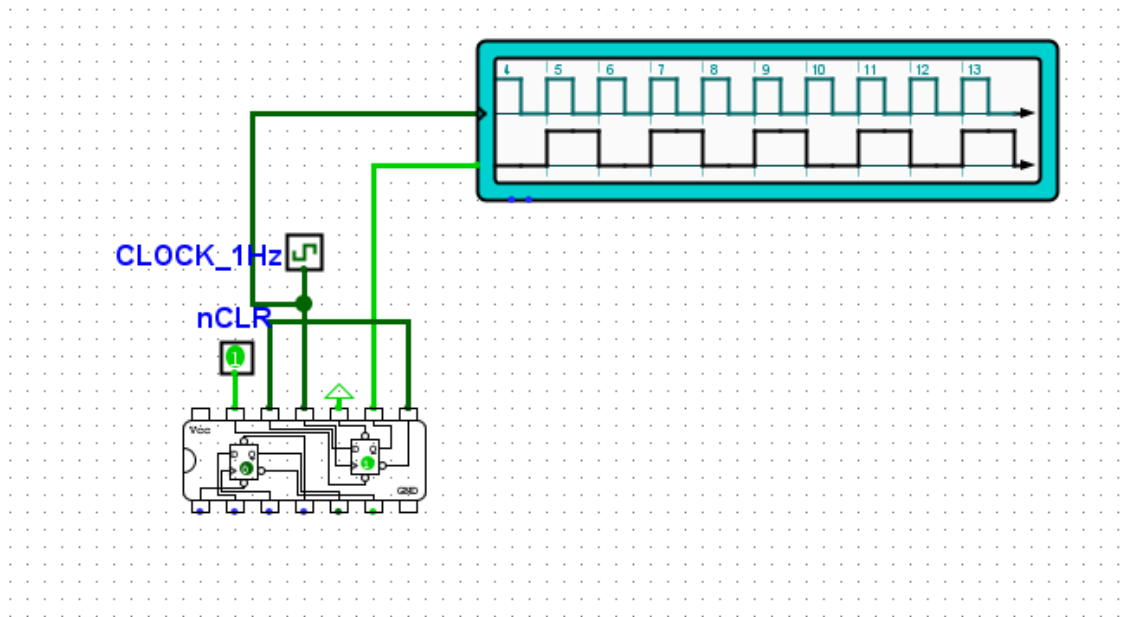
Σχήμα 1. Κύκλωμα Διαίρεσης Συχνότητας Τετραγωνικού Παλμού

#### Παρατηρήσεις:

- Χρησιμοποιείτε **ΜΟΝΟ το ολοκληρωμένο 7474** από την βιβλιοθήκη **TTL** και τον παλμογράφο (*Digital oscilloscope*) από την βιβλιοθήκη **Input/Output-Extra**
- **ΠΡΟΣΟΧΗ:** Οι είσοδοι CLEAR(ή *Reset*, *R*, *nCLR*) και PRESET(ή *Set*, *S*, *nPRE*) στο 7474 είναι **Active Low!**
- **ΠΡΟΣΟΧΗ:** Για να ξεκινήσει το Flip-Flop να λειτουργεί σωστά πρέπει πρώτα να θέσετε την είσοδο CLEAR(ή *Reset*, *R*, *nCLR*) στο “0” (**CLEAR=0, εκκαθάριση**) και έπειτα να τη θέσετε στο λογικό “1”. Η είσοδος PRESET(*Set*, *S*, *nPRE*) πρέπει να συνδεθεί εξαρχής στο λογικό “1”
- Επιβεβαιώστε πριν την έναρξη της εξομοίωσης ότι η συχνότητα λειτουργίας (tick frequency) του CLOCK\_1Hz **είναι ίση με 1Hz**. (Συμβουλευτείτε την παρουσίαση για τον simulator που βρίσκεται στα “Έγγραφα” του eclass του εργαστηρίου)

## ΚΥΚΛΩΜΑ

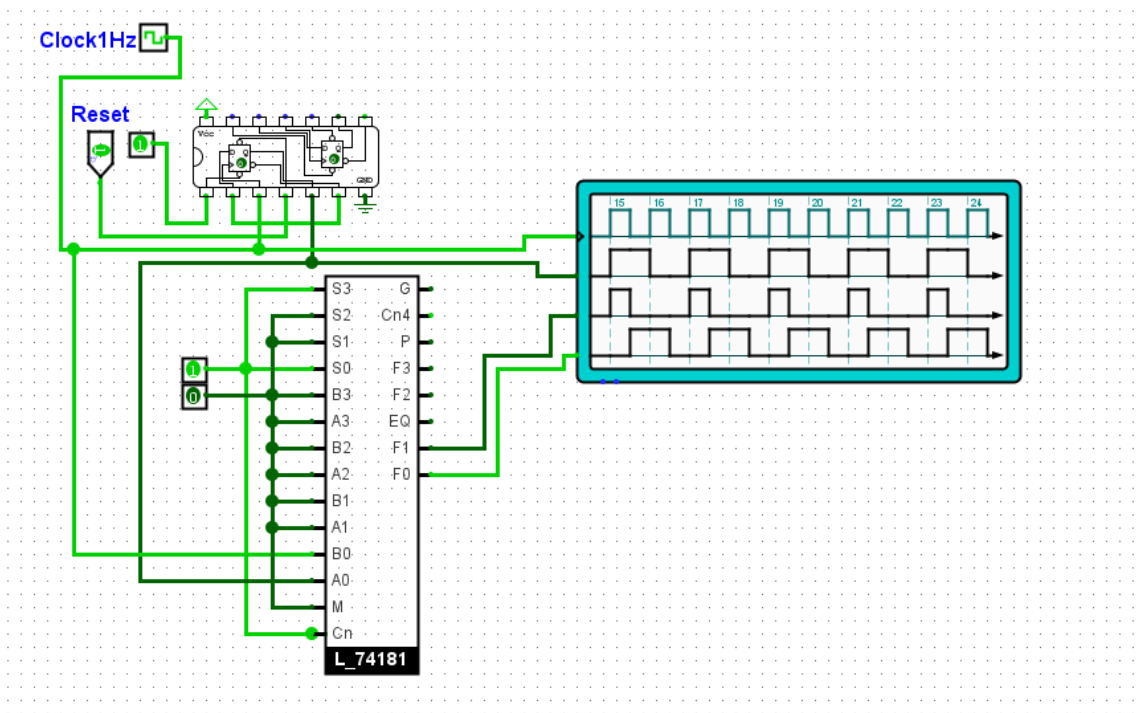
- Screenshot κυκλώματος διαίρεσης συχνότητας (να φαίνονται οι κυματομορφές)



- 3.B.** Χρησιμοποιώντας την ALU (74181) και συνδέοντας τις εισόδους επιλογής της τις κατάλληλες τιμές, υλοποιείτε στον simulator κύκλωμα που να κάνει "*ΑΡΙΘΜΗΤΙΚΗ ΠΡΟΣΘΕΣΗ*" (PLUS) δύο αριθμών του **1-bit**. Τροφοδοτείτε τις εισόδους A<sub>0</sub> και B<sub>0</sub> της ALU με τους δύο τετραγωνικούς παλμούς (με λόγο συχνότητας 1/2 ( $f_1/f_0 = 1/2$ )) από το κύκλωμα του Ερωτ. 3.A, αντίστοιχα. Βεβαιωθείτε για τη σωστή λειτουργία της ALU ελέγχοντας τις εξόδους F<sub>0</sub> και F<sub>1</sub> με τον παλμογράφο.

## Παρατηρήσεις:

- Από την βιβλιοθήκη **CEID\_LogicDesign\_LogisimEV** χρησιμοποιείτε **MONO** το ολοκληρωμένο 74181. Από την βιβλιοθήκη **TTL MONO το ολοκληρωμένο 7474** και από την βιβλιοθήκη **Input/Output-Extra**, τον παλμογράφο (Digital oscilloscope).
- Ρυθμίστε τον παλμογράφο έτσι ώστε να απεικονίζει συνολικά τα 4 σήματα CLOCK\_1Hz, Q, F<sub>1</sub>, F<sub>0</sub>, και να δείχνει όλους τους δυνατούς συνδυασμούς των 2 εισόδων A<sub>0</sub> και B<sub>0</sub> της ALU.

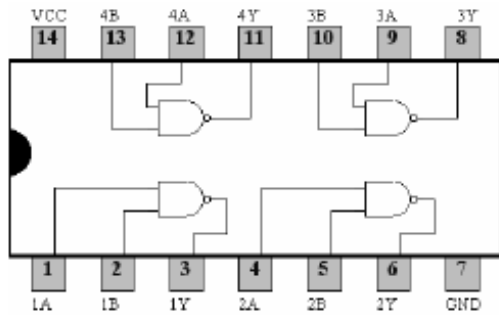
**ΚΥΚΛΩΜΑ (screenshot) όπου φαίνονται οι κυματομορφές εξόδου (όλοι οι συνδυασμοί)****Σύντομη Δικαιολόγηση των κυματομορφών**

Διακρίνουμε 4 διαφορετικές περιπτώσεις που επαναλαμβάνονται συναρτήσει του Clock1Hz. Στην πρώτη περίπτωση, και οι δύο εισοδοι A0 και B0 είναι 1, άρα  $1 + 1 = 10$  στην έξοδο άρα η F1 δίνει σήμα 1 και η F0 δίνει σήμα 0. Στην δεύτερη περίπτωση, η είσοδος A0 είναι 1 ενώ η B0 0, συνεπώς  $1 + 0 = 1$ , άρα η F1 δίνει σήμα 0 και η F0 δίνει σήμα 1. Η τρίτη περίπτωση είναι ίδια με την δεύτερη, με αντεστραμμένο τις εισόδους A0 και B0. Στην τέταρτη περίπτωση, και οι δύο εισοδοι είναι 0 συνεπώς και οι δύο έξοδοι δίνουν σήματα 0.

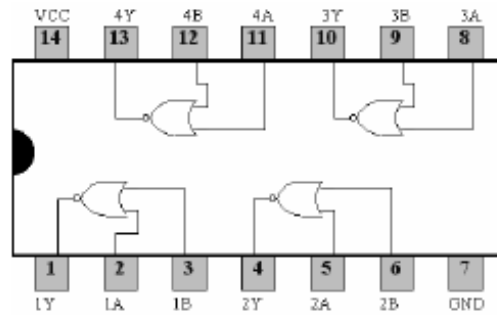
**ΒΟΗΘΗΤΙΚΗ ΣΕΛΙΔΑ.**

## ΠΑΡΑΡΤΗΜΑ

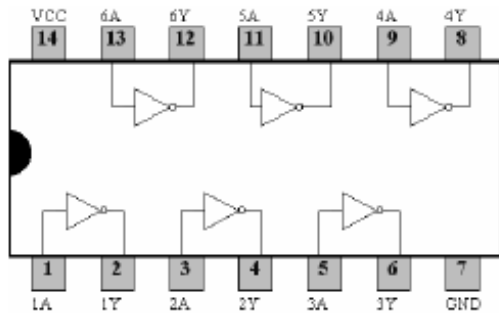
### Διαγράμματα Ολοκληρωμένων



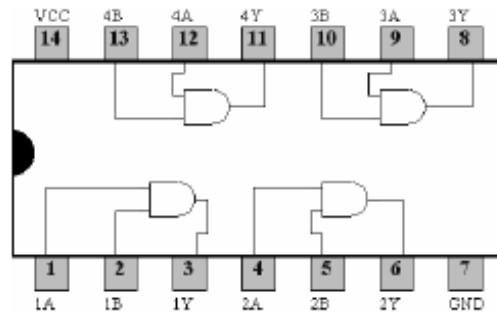
7400 Quad 2-Input NAND Gate



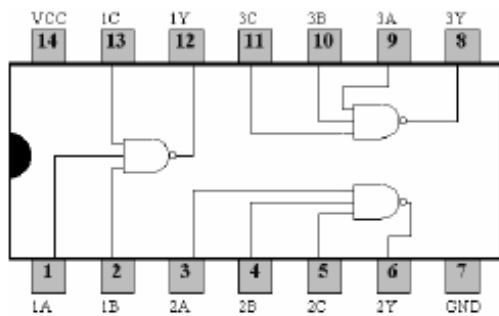
7402 Quad 2-Input NOR Gate



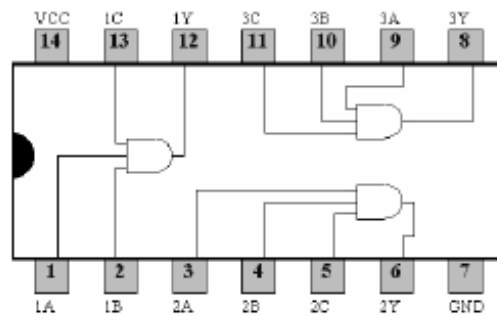
7404 Hex Inverter



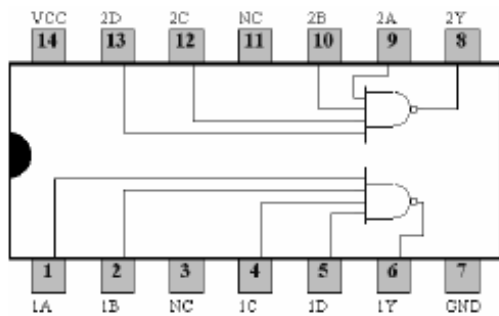
7408 Quad 2-Input AND Gate



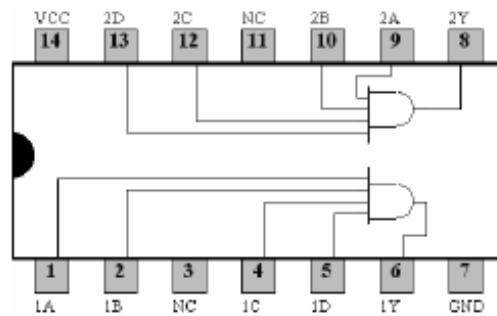
7410 Triple 3-Input NAND Gate



7411 Triple 3-Input AND Gate

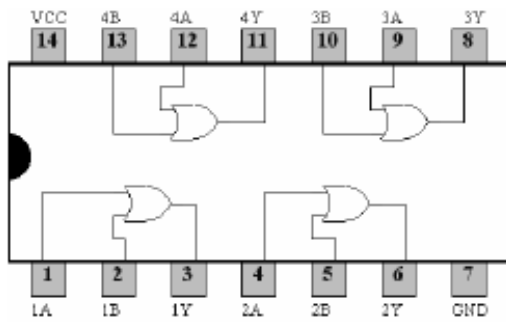


7420 Dual 4-Input NAND Gate

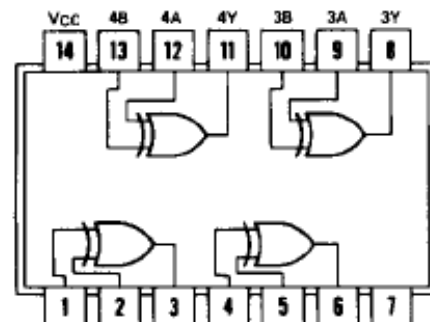


7421 Dual 4-Input AND Gate

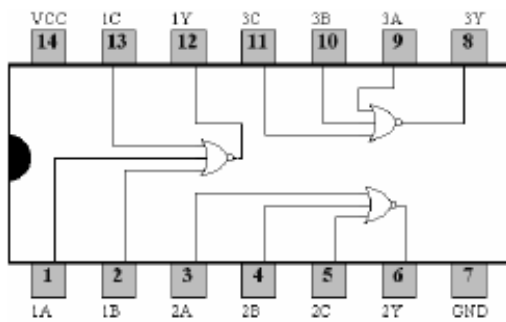




7432 Quad 2-Input OR Gate



7486 Quad 2-Input XOR Gate

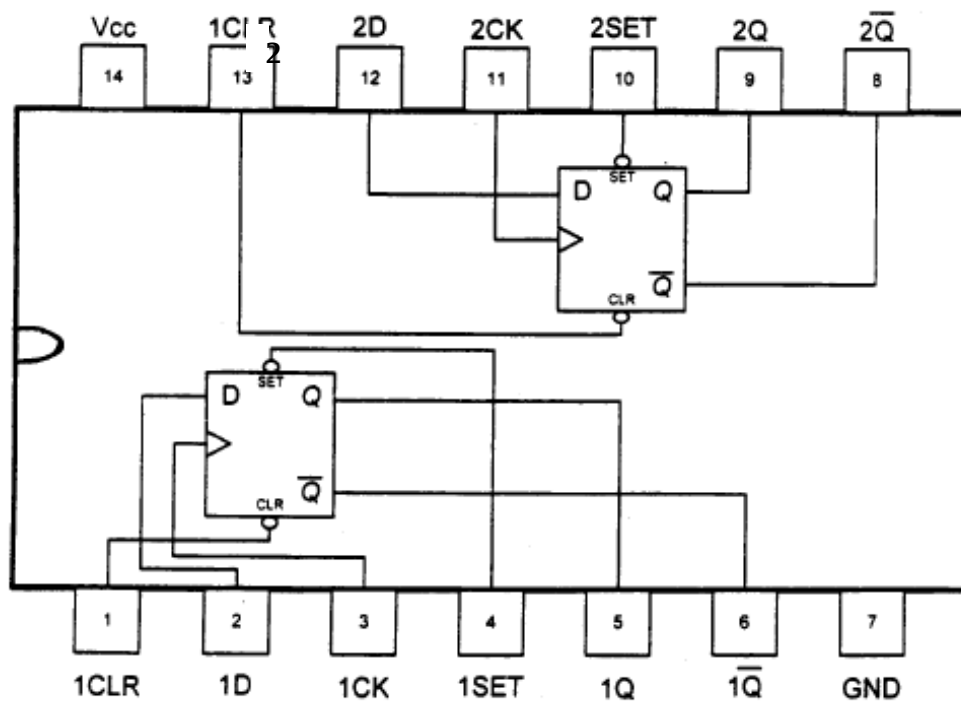


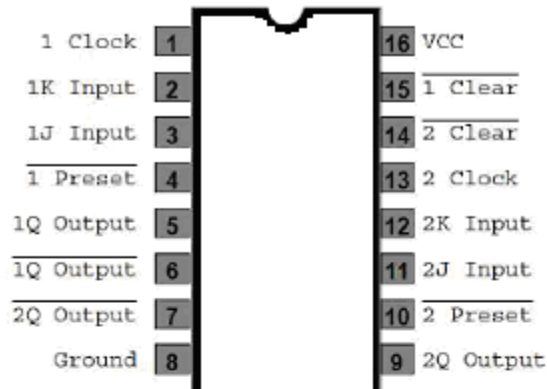
7427 Triple 3-Input NOR Gate

## DUAL D-TYPE POSITIVE EDGE TRIGGERED FLIP-FLOPS WITH PRESET AND CLEAR

FUNCTION TABLE

| INPUTS |       |       |   | OUTPUTS        |                 |
|--------|-------|-------|---|----------------|-----------------|
| PRESET | CLEAR | CLOCK | D | Q              | Q'              |
| L      | H     | X     | X | H              | L               |
| H      | L     | X     | X | L              | H               |
| L      | L     | X     | X | H              | H               |
| H      | H     | ↑     | H | H              | L               |
| H      | H     | ↑     | L | L              | H               |
| H      | H     | L     | X | Q <sub>0</sub> | Q' <sub>0</sub> |



**DUAL JK FLIP\_FLOP with CLEAR/PRESET****74112****Function Table**

| Inputs |     |     |   |   | Outputs    |                  |
|--------|-----|-----|---|---|------------|------------------|
| PR     | CLR | CLK | J | K | Q          | $\overline{Q}$   |
| L      | H   | X   | X | X | H          | L                |
| H      | L   | X   | X | X | L          | H                |
| L      | L   | X   | X | X | H (Note 1) | H (Note 1)       |
| H      | H   | ↓   | L | L | $Q_0$      | $\overline{Q}_0$ |
| H      | H   | ↓   | H | L | H          | L                |
| H      | H   | ↓   | L | H | L          | H                |
| H      | H   | ↓   | H | H | Toggle     |                  |
| H      | H   | H   | X | X | $Q_0$      | $\overline{Q}_0$ |

H = HIGH Logic Level

L = LOW Logic Level

X = Either LOW or HIGH Logic Level

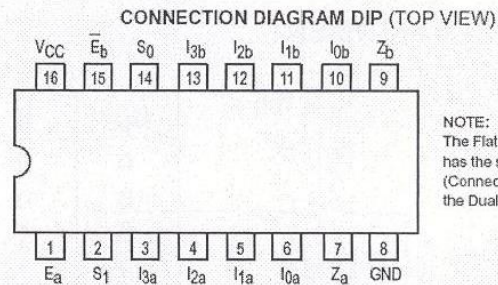
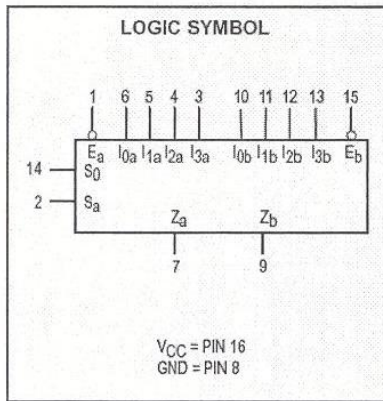
↓ = Negative Going Edge of Pulse

 $Q_0$  = The output logic level before the indicated input conditions were established.

Toggle = Each output changes to the complement of its previous level on each falling edge of the clock pulse.

**Note 1:** This configuration is nonstable; that is, it will not persist when preset and/or clear inputs return to their inactive (HIGH) level.

74153

**DUAL 4-INPUT MULTIPLEXER**

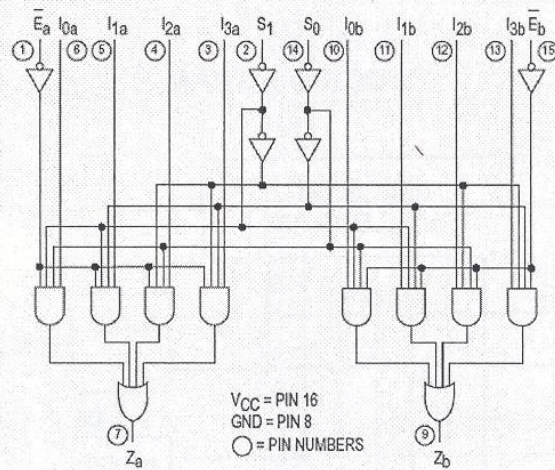
NOTE:  
The Flatpak version  
has the same pinouts  
(Connection Diagram) as  
the Dual In-Line Package.

**PIN NAMES**

$S_0$  Common Select Input  
 $E$  Enable (Active LOW) Input  
 $I_0, I_1$  Multiplexer Inputs  
 $Z$  Multiplexer Output (Note b)

**LOADING (Note a)**

| HIGH     | LOW          |
|----------|--------------|
| 0.5 U.L. | 0.25 U.L.    |
| 0.5 U.L. | 0.25 U.L.    |
| 0.5 U.L. | 0.25 U.L.    |
| 10 U.L.  | 5 (2.5) U.L. |

**LOGIC DIAGRAM****TRUTH TABLE**

| SELECT INPUTS |       | INPUTS (a or b) |       |       |       |       |  | OUTPUT |
|---------------|-------|-----------------|-------|-------|-------|-------|--|--------|
| $S_0$         | $S_1$ | $E$             | $I_0$ | $I_1$ | $I_2$ | $I_3$ |  | $Z$    |
| X             | X     | H               | X     | X     | X     | X     |  | L      |
| L             | L     | L               | L     | X     | X     | X     |  | L      |
| L             | L     | L               | H     | X     | X     | X     |  | H      |
| H             | L     | L               | X     | L     | X     | X     |  | L      |
| H             | L     | L               | X     | H     | X     | X     |  | H      |
| L             | H     | L               | X     | X     | L     | X     |  | L      |
| L             | H     | L               | X     | X     | H     | X     |  | H      |
| H             | H     | L               | X     | X     | X     | L     |  | L      |
| H             | H     | L               | X     | X     | X     | H     |  | H      |

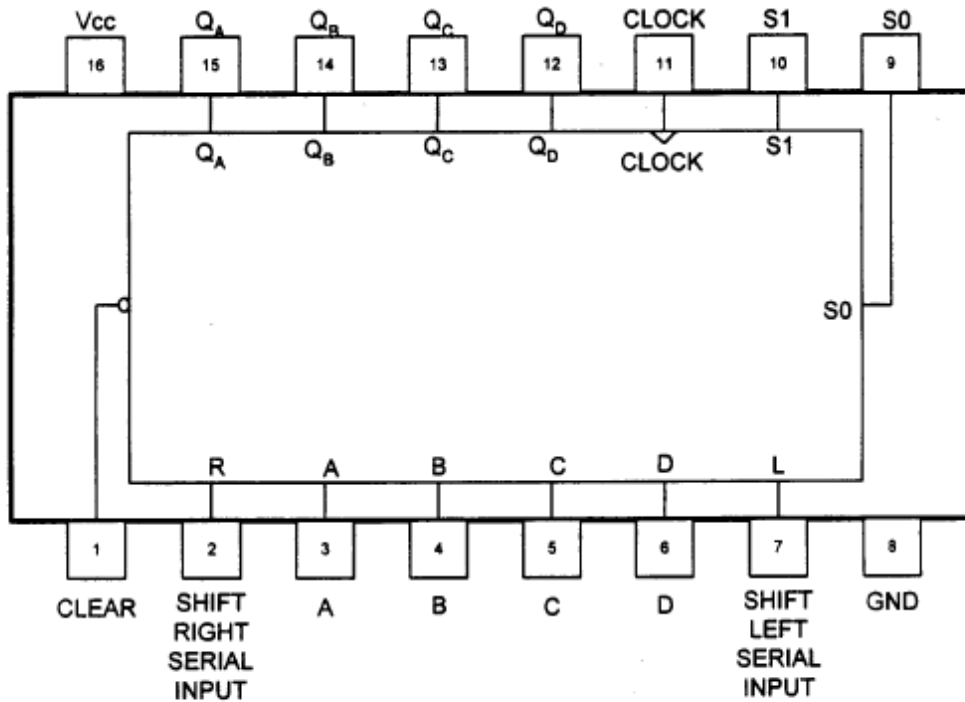
H = HIGH Voltage Level  
L = LOW Voltage Level  
X = Don't Care

74194

## 4 - BIT BIDIRECTIONAL UNIVERSAL SHIFT REGISTER

FUNCTION TABLE

| INPUTS |                |                |       |        |              |          |             |   |   | OUTPUTS         |                 |                 |                 |
|--------|----------------|----------------|-------|--------|--------------|----------|-------------|---|---|-----------------|-----------------|-----------------|-----------------|
| CLEAR  | MODE           |                | CLOCK | SERIAL |              | PARALLEL |             |   |   | Q <sub>A</sub>  | Q <sub>B</sub>  | Q <sub>C</sub>  | Q <sub>D</sub>  |
|        | S <sub>1</sub> | S <sub>0</sub> |       | LEFT   | RIGHT        | A        | B           | C | D |                 |                 |                 |                 |
|        |                |                |       |        |              |          |             |   |   |                 |                 |                 |                 |
| L      | X              | X              | X     | X      | X            | X        | $\bar{E}_a$ | X | X | L               | L               | L               | L               |
| H      | X              | X              | L     | X      | X            | X        | X           | X | X | Q <sub>A0</sub> | Q <sub>B0</sub> | Q <sub>C0</sub> | Q <sub>D0</sub> |
| H      | H              | H              | ↑     | X      | X            | a        | b           | c | d | a               | b               | c               | d               |
| H      | L              | H              | ↑     | X      | <del>H</del> | X        | X           | X | X | H               | Q <sub>An</sub> | Q <sub>Bn</sub> | Q <sub>Cn</sub> |
| H      | L              | H              | ↑     | X      | L            | X        | X           | X | X | L               | Q <sub>An</sub> | Q <sub>Bn</sub> | Q <sub>Cn</sub> |
| H      | H              | L              | ↑     | H      | X            | X        | X           | X | X | Q <sub>Bn</sub> | Q <sub>Cn</sub> | Q <sub>Dn</sub> | H               |
| H      | H              | L              | ↑     | L      | X            | X        | X           | X | X | Q <sub>Bn</sub> | Q <sub>Cn</sub> | Q <sub>Dn</sub> | L               |
| H      | L              | L              | X     | X      | X            | X        | X           | X | X | Q <sub>A0</sub> | Q <sub>B0</sub> | Q <sub>C0</sub> | Q <sub>D0</sub> |



## DM74LS283

### 4-Bit Binary Adder with Fast Carry

#### General Description

These full adders perform the addition of two 4-bit binary numbers. The sum (Σ) outputs are provided for each bit and the resultant carry (C4) is obtained from the fourth bit. These adders feature full internal look ahead across all four bits. This provides the system designer with partial look-ahead performance at the economy and reduced package count of a ripple-carry implementation.

The adder logic, including the carry, is implemented in its true form meaning that the end-around carry can be accomplished without the need for logic or level inversion.

#### Features

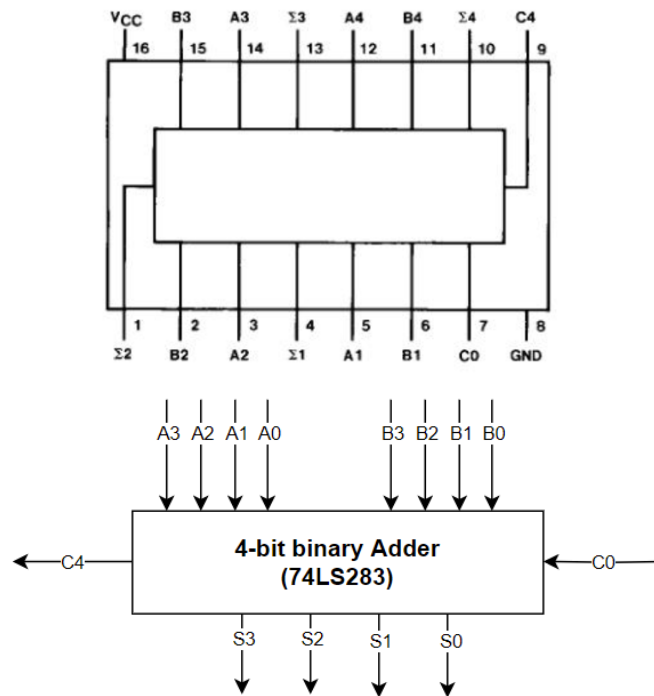
- Full-carry look-ahead across the four bits
- Systems achieve partial look-ahead performance with the economy of ripple carry
- Typical add times
  - Two 8-bit words 25 ns
  - Two 16-bit words 45 ns
- Typical power dissipation per 4-bit adder 95 mW

#### Ordering Code:

| Order Number | Package Number | Package Description                                                          |
|--------------|----------------|------------------------------------------------------------------------------|
| DM74LS283M   | M16A           | 16-Lead Small Outline Integrated Circuit (SOIC), J EDEC MS-012, 0.150 Narrow |
| DM74LS283N   | N16E           | 16-Lead Plastic Dual-In-Line Package (PDIP), J EDEC MS-001, 0.300 Wide       |

Devices also available in Tape and Reel. Specify by appending the suffix letter "X" to the ordering code.

#### Connection Diagram



## 74LS181 – 4-bit ARITHMETIC LOGIC UNIT

TABLE 2

| SELECTION |    |    |    | ACTIVE-HIGH DATA            |                                           |                                                           |
|-----------|----|----|----|-----------------------------|-------------------------------------------|-----------------------------------------------------------|
|           |    |    |    | M = H<br>LOGIC<br>FUNCTIONS | M = L; ARITHMETIC OPERATIONS              |                                                           |
| S3        | S2 | S1 | S0 |                             | $\overline{C}_n = H$<br>(no carry)        | $\overline{C}_n = L$<br>(with carry)                      |
| L         | L  | L  | L  | $F = \overline{A}$          | $F = A$                                   | $F = A \text{ PLUS } 1$                                   |
| L         | L  | L  | H  | $F = \overline{A + B}$      | $F = A + B$                               | $F = (A + B) \text{ PLUS } 1$                             |
| L         | L  | H  | L  | $F = \overline{AB}$         | $F = A + \overline{B}$                    | $F = (A + \overline{B}) \text{ PLUS } 1$                  |
| L         | L  | H  | H  | $F = 0$                     | $F = \text{MINUS } 1 \text{ (2's COMPL)}$ | $F = \text{ZERO}$                                         |
| L         | H  | L  | L  | $F = \overline{AB}$         | $F = A \text{ PLUS } \overline{AB}$       | $F = A \text{ PLUS } \overline{AB} \text{ PLUS } 1$       |
| L         | H  | L  | H  | $F = \overline{B}$          | $F = (A + B) \text{ PLUS } \overline{AB}$ | $F = (A + B) \text{ PLUS } \overline{AB} \text{ PLUS } 1$ |
| L         | H  | H  | L  | $F = A \oplus B$            | $F = A \text{ MINUS } B \text{ MINUS } 1$ | $F = A \text{ MINUS } B$                                  |
| L         | H  | H  | H  | $F = \overline{AB}$         | $F = \overline{AB} \text{ MINUS } 1$      | $F = \overline{AB}$                                       |
| H         | L  | L  | L  | $F = \overline{A + B}$      | $F = A \text{ PLUS } AB$                  | $F = A \text{ PLUS } AB \text{ PLUS } 1$                  |
| H         | L  | L  | H  | $F = A \oplus B$            | $F = A \text{ PLUS } B$                   | $F = A \text{ PLUS } B \text{ PLUS } 1$                   |
| H         | L  | H  | L  | $F = B$                     | $F = (A + \overline{B}) \text{ PLUS } AB$ | $F = (A + \overline{B}) \text{ PLUS } AB \text{ PLUS } 1$ |
| H         | L  | H  | H  | $F = AB$                    | $F = AB \text{ MINUS } 1$                 | $F = AB$                                                  |
| H         | H  | L  | L  | $F = 1$                     | $F = A \text{ PLUS } A^\dagger$           | $F = A \text{ PLUS } A \text{ PLUS } 1$                   |
| H         | H  | L  | H  | $F = A + \overline{B}$      | $F = (A + B) \text{ PLUS } A$             | $F = (A + B) \text{ PLUS } A \text{ PLUS } 1$             |
| H         | H  | H  | L  | $F = A + B$                 | $F = (A + \overline{B}) \text{ PLUS } A$  | $F = (A + \overline{B}) \text{ PLUS } A \text{ PLUS } 1$  |
| H         | H  | H  | H  | $F = A$                     | $F = A \text{ MINUS } 1$                  | $F = A$                                                   |

<sup>†</sup> Each bit is shifted to the next more significant position.

